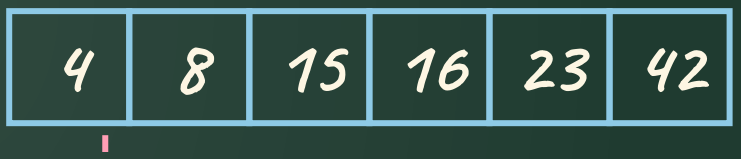
PrepPath E-books Store



DSA with Java

Handwritten Chalkboard Notes for Beginners & Interviews

- ✦ Hand-drawn diagrams for every idea
- ✦ Java code compiled & tested
- ✦ Real-world logic problems
- ✦ Patterns, 30-day plan & web references

Learn it. Draw it. Code it.

START HERE

Contents & Roadmap

These are **chalkboard notes** for learning **DSA with Java**: simple words, a picture for every idea, and **Java code that is compiled and tested**. Each topic ends with the real-world logic problems that interviewers love.



Learning order, left to right.

Chapter	What you learn	Page
Java Toolkit	Syntax, collections cheat sheet	3
Big-O	Speed, growth, space	5
Arrays	Prefix sum, Kadane, rotate	7
Strings	StringBuilder, palindromes	10
Two Pointers & Windows	Pairs, sliding window	12
Hashing	HashMap, HashSet, counting	15
Recursion	Think small, call stack	17
Sorting	Basic sorts to quick sort, Comparator	20
Binary Search	Halving the search	24
Linked List	Nodes, reverse, cycles	27
Stack	LIFO, monotonic stack	31
Queue & Deque	FIFO, sliding window max	34
Trees & BST	Traversals, LCA	36
Heap	PriorityQueue, top K	41
Graphs	BFS, DFS, Dijkstra, DSU	43
Greedy	Best choice now	49
Backtracking	Try, undo, try again	51
Dynamic Programming	Remember sub-answers	53
Bit Tricks	XOR and masks	56
Interview Guide	Framework, patterns, plan	57

JAVA BASICS

Java Refresher for DSA

Java is **strongly typed**: every variable has a type. Every program starts in `main`. These basics appear in every DSA solution.

Type	Size	Range / use
int	32 bit	about ±2.1 billion, the default whole number
long	64 bit	huge sums, factorials (use suffix L)
double	64 bit	decimals (3.14)
char	16 bit	one character 'a' (acts like a number)
boolean	1 bit	true / false

things that surprise beginners

```
static void demo() {
    int a = 7, b = 2;
    System.out.println(a / b);            // 3    integer division drops decimals!
    System.out.println(a / (double) b);   // 3.5
    System.out.println(a % b);            // 1    remainder
    int big = Integer.MAX_VALUE;
    System.out.println(big + 1);          // overflow wraps to negative!
    System.out.println('a' + 1);          // 98   char acts like a number
}
```

output:

```
3
3.5
1
-2147483648
98
```

loops: two styles

```
static int sum(int[] arr) {
    int total = 0;
    for (int x : arr) total += x;          // for-each loop
    return total;
}
static int sumIndexed(int[] arr) {
    int total = 0;
    for (int i = 0; i < arr.length; i++) total += arr[i];   // arr.length (no brackets!)
    return total;
}
```

Two classic traps

Interview habit: when sums may exceed 2 billion, use `long`. Compare strings with `.equals()`, never `==`.

JAVA BASICS

Java Collections: Your DSA Toolbox

Java already ships the data structures. In interviews you *use* them, so learn which class to pick.

Need	Class	Key methods
Resizable array	ArrayList	add, get, set, size, remove
Key → value	HashMap	put, get, getOrDefault, containsKey
Unique items	HashSet	add, contains, remove
Stack / Queue / Deque	ArrayDeque	push, pop, offer, poll, peek
Smallest/largest first	PriorityQueue	add, poll, peek
Sorted map / set	TreeMap / TreeSet	firstKey, floorKey, ceilingKey

one line of each (generics <> hold the type)

```
static int demo() {
    List<Integer> list = new ArrayList<>();   list.add(5); list.add(9);
    Map<String, Integer> map = new HashMap<>(); map.put("a", 1);
    Set<Integer> set = new HashSet<>();         set.add(3); set.add(3);   //
duplicate ignored
    Deque<Integer> stack = new ArrayDeque<>(); stack.push(1);
stack.push(2);
    Queue<Integer> queue = new ArrayDeque<>(); queue.offer(1);
queue.offer(2);
    PriorityQueue<Integer> pq = new PriorityQueue<>(); pq.add(8);
pq.add(2);
    return list.get(1) + map.getOrDefault("b", 0) + set.size() +
stack.pop() + queue.poll() + pq.poll();
}
```

Pro tip

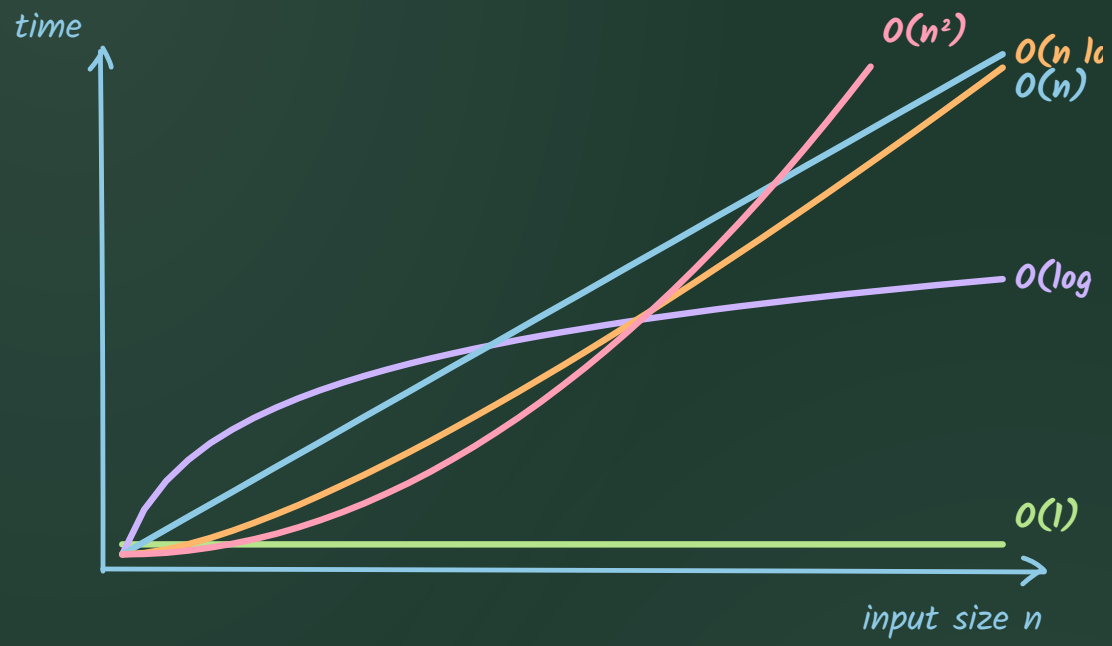
Use `ArrayDeque`, *not* `Stack` / `LinkedList`: it is faster and the modern choice for stack, queue and deque. `List<Integer>` holds objects, so `int[]` is faster for big data.

- `int[]` array: fixed size, fastest.
- `ArrayList<Integer>`: grows automatically (autoboxing int ↔ Integer).
- Interfaces on the left, class on the right: `Map<K,V> m = new HashMap<>();`

BIG-O

Big-O: How Fast Is My Code?

Big-O measures how the work grows as the input size **n** grows. It ignores machine speed and small details. Interviewers always ask: *what is the time and space complexity?*



Common growth rates. Flatter is faster.

count the steps for n = 16

```
static void demo(int n) {
    int steps = 0;
    for (int i = 0; i < n; i++) steps++;                  // O(n)
    System.out.println("single loop: " + steps);
    steps = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) steps++;              // O(n^2)
    System.out.println("nested loops: " + steps);
    steps = 0;
    for (int i = n; i > 1; i /= 2) steps++;               // O(log n)
    System.out.println("halving: " + steps);
}
```

output:
```
single loop: 16
nested loops: 256
halving: 4
```

Simplify

Rules: drop constants (3n → n), keep the biggest term (n² + n → n²), loops in a row **add**, loops inside loops **multiply**.

BIG-O

Complexity Cheat Sheet for Java

Operation	Time
ArrayList get(i) / set	O(1)
ArrayList add at end	O(1) amortised
ArrayList middle add/remove, contains	O(n)
HashMap / HashSet get, put, contains	O(1) average
TreeMap / TreeSet operations	O(log n)
PriorityQueue add / poll	O(log n)
PriorityQueue peek	O(1)
ArrayDeque push, pop, offer, poll	O(1)
Arrays.sort / Collections.sort	O(n log n)

Input size guide: n ≈ 1,000 → $O(n^2)$ ok. n ≈ 100,000 → O(n log n). n ≥ 1,000,000 → O(n).

Same problem, two speeds: two-sum

```
static int[] twoSumSlow(int[] a, int target) {        // O(n^2) time, O(1)
space
    for (int i = 0; i < a.length; i++)
        for (int j = i + 1; j < a.length; j++)
            if (a[i] + a[j] == target) return new int[]{i, j};
    return new int[]{};
}
static int[] twoSumFast(int[] a, int target) {        // O(n) time, O(n)
space
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < a.length; i++) {
        Integer j = seen.get(target - a[i]);
        if (j != null) return new int[]{j, i};
        seen.put(a[i], i);
    }
    return new int[]{};
}
```

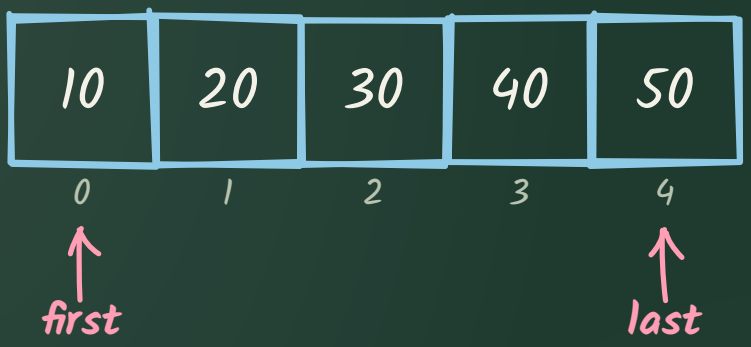
Big idea

Trade-off: we bought speed with memory. Ask yourself: *can extra space remove a loop?*

ARRAYS

Arrays: The Foundation

An *array* is a row of boxes in memory with the *same type* and *fixed size*. Access by index is instant: *O(1)*.



Indexes start at 0. Last index = length − 1.

```
static void demo() {
    int[] a = {5, 3, 8};                // create with values
    int[] b = new int[4];               // zeros: {0, 0, 0, 0}
    int[][] grid = new int[2][3];       // 2 rows, 3 columns
    grid[1][2] = 7;
    System.out.println(Arrays.toString(a));
    System.out.println(Arrays.deepToString(grid));
    int[] c = Arrays.copyOf(a, 5);      // new array, longer, padded with
0
    Arrays.fill(b, 9);
    Arrays.sort(a);
    System.out.println(Arrays.toString(a) + " " + Arrays.toString(c) + " "
+ Arrays.toString(b));
}
```

output:
```
[5, 3, 8]
[[0, 0, 0], [0, 0, 7]]
[3, 5, 8] [5, 3, 8, 0, 0] [9, 9, 9, 9]
```

Operation	Cost
read / write a[i]	O(1)
search (unsorted)	O(n)
insert / delete in the middle	O(n) shifting

Don't mix up

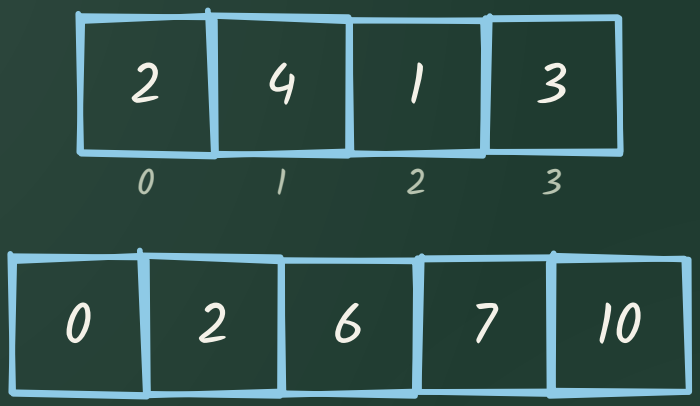
`a.length` for arrays, `s.length()` for Strings, `list.size()` for lists. Different on purpose!

ARRAYS

Arrays: Prefix Sum & Kadane

Prefix sum: answer range queries instantly

Build `p[i]` = sum of the first i numbers once. Then *sum(l..r) = p[r+1] − p[l]* in O(1).



Top: array. Bottom: prefix sums. Sum of index 1..3 = 10 − 2 = 8.

```java
static int[] prefix(int[] a) {
    int[] p = new int[a.length + 1];
    for (int i = 0; i < a.length; i++) p[i + 1] = p[i] + a[i];
    return p;
}
static int rangeSum(int[] p, int l, int r) { return p[r + 1] - p[l]; }
```

Kadane: maximum subarray sum

Walk once. At each number choose: ***extend*** the current run or ***start fresh*** here. Keep the best seen.

O(n) time, O(1) space

```java
static int kadane(int[] a) {
    int best = a[0], cur = a[0];
    for (int i = 1; i < a.length; i++) {
        cur = Math.max(a[i], cur + a[i]);      // extend or restart
        best = Math.max(best, cur);
    }
    return best;
}
```

Edge case

Kadane works even when ***all numbers are negative***: that is why we start with `a[0]`, not 0.

ARRAYS

Arrays: Classic Interview Problems

1. Reverse in place (two ends move inward)

```
static void reverse(int[] a, int l, int r) {
    while (l < r) { int t = a[l]; a[l++] = a[r]; a[r--] = t; }
}
```

2. Rotate right by k: reverse three times

```
static void rotate(int[] a, int k) {
    k %= a.length;
    reverse(a, 0, a.length - 1);      // whole array
    reverse(a, 0, k - 1);             // first k
    reverse(a, k, a.length - 1);      // the rest
}
```

3. Move zeroes to the end (keep order)

```
static void moveZeroes(int[] a) {
    int w = 0;                                  // next write position
    for (int x : a) if (x ≠ 0) a[w++] = x;
    while (w < a.length) a[w++] = 0;
}
```

4. Missing number in 0..n

```
static int missing(int[] a) {
    int n = a.length, expected = n * (n + 1) / 2, actual = 0;
    for (int x : a) actual += x;
    return expected - actual;
}
```

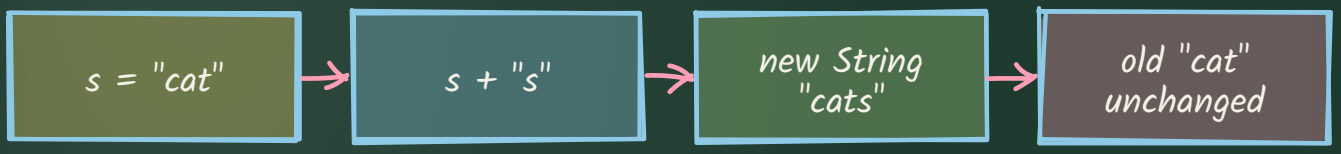
Remember

Pattern: *in-place + one pass + a write pointer* solves many array problems in O(n) time and O(1) space.

STRINGS

Strings in Java

A Java `String` is ***immutable***: once created it never changes. Every `+` or `replace` makes a ***new*** String. For many edits use ***StringBuilder***.



Strings never change in place.

```
static void demo() {
    String s = "hello world";
    System.out.println(s.length() + " " + s.charAt(4) + " " +
s.indexOf("world"));
    System.out.println(s.substring(0, 5) + "|" + s.toUpperCase() + "|" +
s.replace('l', 'L'));
    System.out.println(Arrays.toString(s.split(" ")) + " " +
s.contains("lo") + " " + s.equals("hello world"));
    char[] cs = s.toCharArray();              // handy for in-place work
    Arrays.sort(cs);
    System.out.println(new String(cs).trim());
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < 3; i++) sb.append(i).append(',');
    sb.reverse();
    System.out.println(sb);
}
```

output:
```
11 o 6
hello|HELLO WORLD|heLLo worLd
[hello, world] true true
dehlllloorw
,2,1,0
```

Task	Use
compare content	s.equals(t) / equalsIgnoreCase
many appends	StringBuilder (O(1) each)
char ↔ int	c - 'a' gives 0..25

Interview trap

`s += x` inside a loop of n steps costs $O(n^2)$ because each `+` copies the whole string. Use `StringBuilder`.

STRINGS

Strings: Real Interview Problems

1. Is it a palindrome? (ignore case & symbols)

```
static boolean isPalindrome(String s) {
    int l = 0, r = s.length() - 1;
    while (l < r) {
        while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
        while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
        if (Character.toLowerCase(s.charAt(l++)) ≠
Character.toLowerCase(s.charAt(r--))) return false;
    }
    return true;
}
```

2. Anagram check with a 26-box counter

```
static boolean isAnagram(String a, String b) {
    if (a.length() ≠ b.length()) return false;
    int[] cnt = new int[26];
    for (int i = 0; i < a.length(); i++) {
        cnt[a.charAt(i) - 'a']++;
        cnt[b.charAt(i) - 'a']--;
    }
    for (int c : cnt) if (c ≠ 0) return false;
    return true;
}
```

3. First non-repeating character

```
static int firstUnique(String s) {
    int[] cnt = new int[26];
    for (char c : s.toCharArray()) cnt[c - 'a']++;
    for (int i = 0; i < s.length(); i++) if (cnt[s.charAt(i) - 'a'] == 1)
return i;
    return -1;
}
```

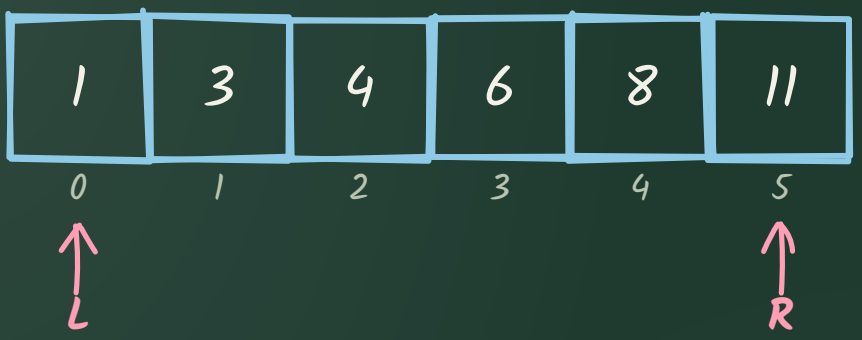
Tip

Counting letters with `int[26]` is faster and simpler than a HashMap when the input is only a-z.

WINDOWS

Two Pointers

Use **two indexes** that walk through the array instead of nested loops. Turns **O(n²)** into **O(n)**. Works best on **sorted** data or when comparing both ends.



Target 10: 1 + 11 = 12 too big, so move R left.

```
static boolean pairSum(int[] a, int target) {          // a is sorted
    int l = 0, r = a.length - 1;
    while (l < r) {
        int s = a[l] + a[r];
        if (s == target) return true;
        if (s < target) l++;        // need bigger: move left pointer right
        else r--;                   // need smaller: move right pointer left
    }
    return false;
}
```

Container with most water

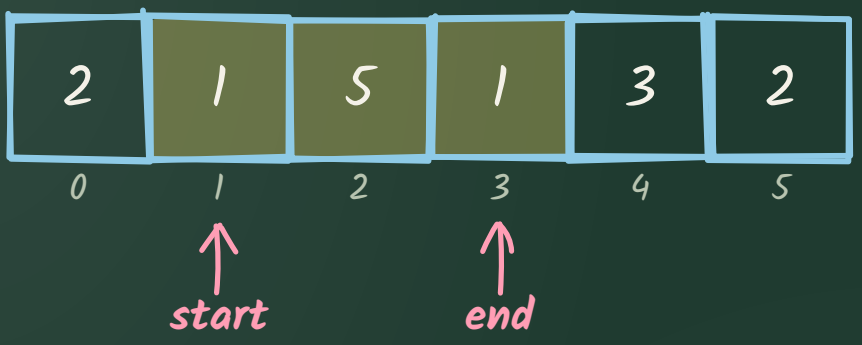
Two walls at the ends. Area = **min(height) × width**. Always move the **shorter** wall, since the other cannot help.

```
static int maxArea(int[] h) {
    int l = 0, r = h.length - 1, best = 0;
    while (l < r) {
        best = Math.max(best, Math.min(h[l], h[r]) * (r - l));
        if (h[l] < h[r]) l++; else r--;
    }
    return best;
}
```

WINDOWS

Sliding Window: Fixed Size

A **window** is a block of k neighbours. Instead of re-adding k numbers each time, **slide**: add the new one on the right, remove the old one on the left. **O(n)**.



Window of size 3 slides one step at a time.

```
static int maxSumK(int[] a, int k) {
    int sum = 0;
    for (int i = 0; i < k; i++) sum += a[i];       // first window
    int best = sum;
    for (int i = k; i < a.length; i++) {
        sum += a[i] - a[i - k];                     // add new, drop old
        best = Math.max(best, sum);
    }
    return best;
}
```

Real-world: highest average of k days

```
static double maxAverage(int[] sales, int k) {
    return (double) maxSumK(sales, k) / k;
}
```

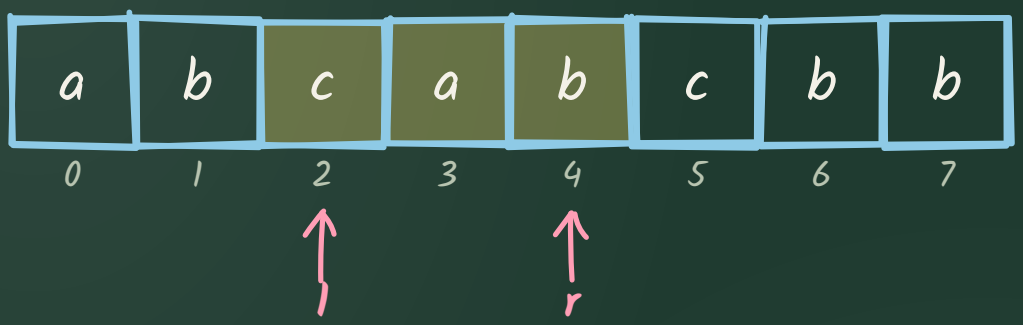
Clue

Spot it: 'subarray / substring of size **k**' or 'every k consecutive' → fixed window.

WINDOWS

Sliding Window: Variable Size

Window size **changes**: grow the right side `r` to add items; when the rule breaks, shrink the left side `l`. Each pointer moves forward only, so the total work is **O(n)**.



Longest substring with no repeated letter: window "cab" has size 3.

```
static int longestUnique(String s) {
    int[] last = new int[128];            // last index where each char
was seen
    Arrays.fill(last, -1);
    int best = 0, l = 0;
    for (int r = 0; r < s.length(); r++) {
        char c = s.charAt(r);
        if (last[c] ≥ l) l = last[c] + 1;  // repeat inside window: jump l
past it
        last[c] = r;
        best = Math.max(best, r - l + 1);
    }
    return best;
}
```

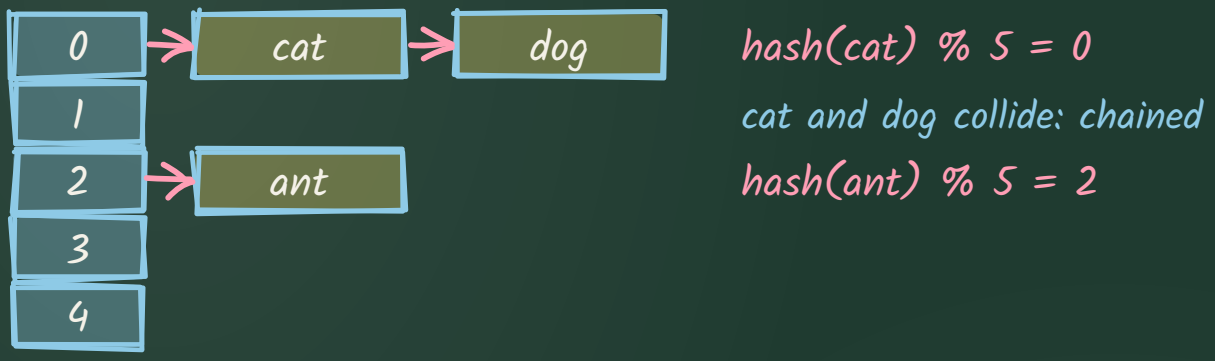
Smallest subarray with sum ≥ target

```
static int minLen(int target, int[] a) {
    int l = 0, sum = 0, best = Integer.MAX_VALUE;
    for (int r = 0; r < a.length; r++) {
        sum += a[r];                              // grow
        while (sum ≥ target) {                    // shrink while still valid
            best = Math.min(best, r - l + 1);
            sum -= a[l++];
        }
    }
    return best == Integer.MAX_VALUE ? 0 : best;
}
```

HASHING

Hashing: HashMap & HashSet

A **hash function** turns a key into a bucket number, so lookup is about **O(1)**. Use it for 'have I seen this?' and 'how many times?'



Buckets 0–4. Collisions share a bucket (a short chain).

```
static void demo() {
    Map<Character, Integer> freq = new TreeMap<>();      // TreeMap prints
sorted
    for (char c : "banana".toCharArray())
        freq.put(c, freq.getOrDefault(c, 0) + 1);        // count letters
    System.out.println(freq);
    for (Map.Entry<Character, Integer> e : freq.entrySet())
        System.out.print(e.getKey() + "=" + e.getValue() + " ");
    System.out.println();
    Set<Integer> seen = new HashSet<>();
    System.out.println(seen.add(4) + " " + seen.add(4) + " " +
seen.contains(4));
}
```

```
output: {a=3, b=1, n=2}
a=3 b=1 n=2
true false true
```

Class	Order	Speed
HashMap / HashSet	none	O(1)
LinkedHashMap	insertion order	O(1)
TreeMap / TreeSet	sorted by key	O(log n)

Custom keys

Keys must implement **equals() and hashCode()** consistently. Strings, Integers and Characters already do.

HASHING

Hashing: Real-World Problems

1. Contains duplicate?

```
static boolean hasDuplicate(int[] a) {
    Set<Integer> seen = new HashSet<>();
    for (int x : a) if (!seen.add(x)) return true;   // add returns false
if present
    return false;
}
```

2. Group anagrams

```
static Collection<List<String>> groupAnagrams(String[] words) {
    Map<String, List<String>> map = new HashMap<>();
    for (String w : words) {
        char[] cs = w.toCharArray();
        Arrays.sort(cs);
        String key = new String(cs);                    // "eat" and "tea"
share "aet"
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(w);
    }
    return map.values();
}
```

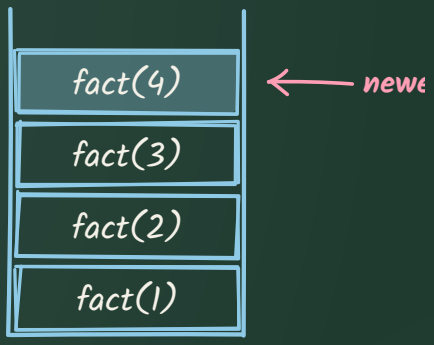
3. Subarray sum equals k (prefix sum + map)

```
static int subarraySum(int[] a, int k) {
    Map<Integer, Integer> count = new HashMap<>();
    count.put(0, 1);                                    // empty prefix
    int sum = 0, ans = 0;
    for (int x : a) {
        sum += x;
        ans += count.getOrDefault(sum - k, 0);          // earlier prefix
that leaves k
        count.merge(sum, 1, Integer::sum);
    }
    return ans;
}
```

RECURSION

Recursion: Think Small

Recursion = a function that calls **itself** on a smaller problem. Two parts: a **base case** (when to stop) and a **recursive case** (shrink and trust the call).



The call stack: each call waits for the one above it to finish.

```java
static int factorial(int n) {
    if (n <= 1) return 1;              // base case
    return n * factorial(n - 1);       // trust the smaller answer
}
static int sumDigits(int n) {
    if (n < 10) return n;
    return n % 10 + sumDigits(n / 10);
}
static void countdown(int n) {
    if (n == 0) { System.out.println("Go!"); return; }
    System.out.print(n + " ");
    countdown(n - 1);
}
```

output: `3 2 1 Go!`

Recipe

3 questions: What is the smallest case? How do I make the problem smaller? What do I do with the smaller answer?

- No base case → **StackOverflowError**.
- Each call uses memory: depth n means **O(n)** stack space.

RECURSION

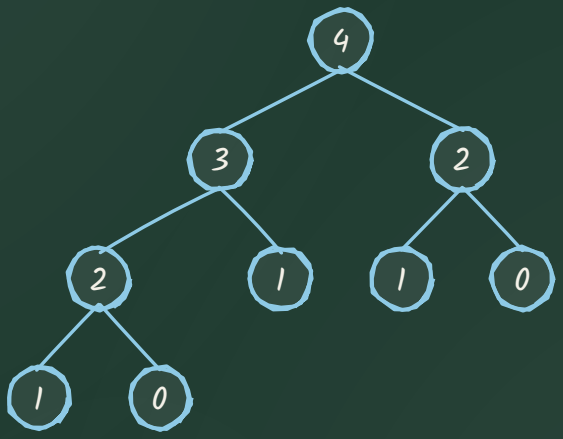
Recursion: Powers & Fibonacci

Fast power: halve the problem

$x^{10} = (x^5)^2$. Halving the exponent gives **O(log n)** instead of O(n).

```
static double power(double x, int n) {
    if (n == 0) return 1;
    if (n < 0) return 1 / power(x, -n);
    double half = power(x, n / 2);
    return n % 2 == 0 ? half * half : half * half * x;
}
```

Fibonacci: the same work done twice



fib(2) and fib(1) repeat. Plain recursion costs about $O(2^n)$.

```
static long fibSlow(int n) { return n < 2 ? n : fibSlow(n - 1) + fibSlow(n
- 2); }   // O(2^n)

static long fib(int n, long[] memo) {
// O(n)
    if (n < 2) return n;
    if (memo[n] != 0) return memo[n];                    // already solved
    return memo[n] = fib(n - 1, memo) + fib(n - 2, memo);
}
```

Preview

Saving answers of repeated calls is called **memoization**, the first step toward **dynamic programming**.

RECURSION

Recursion: Hanoi, Palindrome & Patterns

Tower of Hanoi

Move n disks from A to C using B. To move n: move n−1 to B, move the big one, move n−1 onto it. Total moves = $2^n - 1$.

```
static int hanoi(int n, char from, char to, char via) {
    if (n == 0) return 0;
    int moves = hanoi(n - 1, from, via, to);
    System.out.println("disk " + n + ": " + from + " → " + to);
    return moves + 1 + hanoi(n - 1, via, to, from);
}
```

output:
```
disk 1: A → B
disk 2: A → C
disk 1: B → C
```

Palindrome & reverse with recursion

```
static boolean isPal(String s, int l, int r) {
    if (l ≥ r) return true;
    return s.charAt(l) == s.charAt(r) && isPal(s, l + 1, r - 1);
}
static String rev(String s) { return s.isEmpty() ? "" : rev(s.substring(1))
+ s.charAt(0); }
```

Pattern	Idea
Shrink by 1	factorial, sum of list
Halve	fast power, binary search
Two branches	Fibonacci, subsets → backtracking
Try all choices	permutations, N-Queens

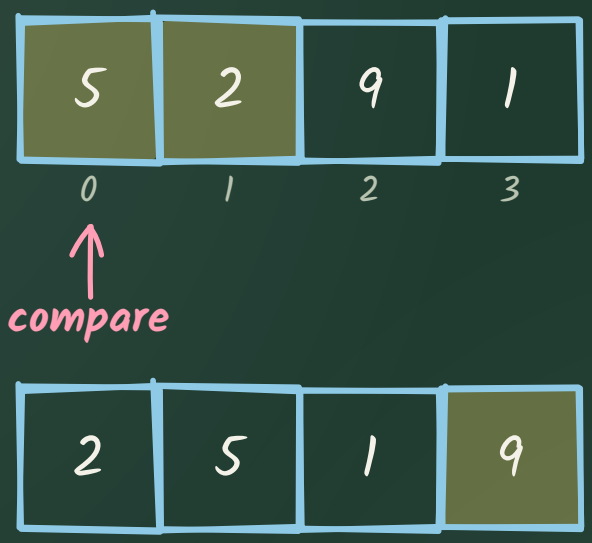
Link

When a recursive solution repeats the *same subproblem*, add memoization. When it needs to *undo* choices, use backtracking.

SORTING

Sorting: Bubble, Selection, Insertion

These are $O(n^2)$ but simple: compare, swap, keep part sorted. (*Selection sort* repeatedly picks the minimum and swaps it to the front.)



Bubble sort: swap out-of-order neighbours; the biggest bubbles to the end.

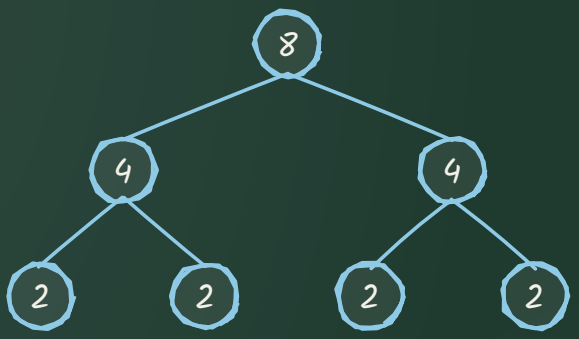
```
static void bubble(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        boolean swapped = false;
        for (int j = 0; j < a.length - 1 - i; j++)
            if (a[j] > a[j + 1]) { int t = a[j]; a[j] = a[j + 1]; a[j + 1]
= t; swapped = true; }
        if (!swapped) break;                // already sorted: O(n) best
case
    }
}
static void insertion(int[] a) {
    for (int i = 1; i < a.length; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; j--; }   // shift
bigger ones right
        a[j + 1] = key;
    }
}
```

Sort	Best	Worst	Stable?
Bubble	O(n)	O(n²)	yes
Insertion	O(n)	O(n²)	yes

SORTING

Merge Sort: Divide & Conquer

Split the array in halves, **sort** each half (recursively), then **merge** the two sorted halves. Always **O(n log n)**, needs O(n) extra space, and is **stable**.



Sizes halve each level (log n levels); each level merges n items in total.

```
static void mergeSort(int[] a, int l, int r) {
    if (l ⩾ r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}
static void merge(int[] a, int l, int m, int r) {
    int[] tmp = new int[r - l + 1];
    int i = l, j = m + 1, k = 0;
    while (i ⩽ m && j ⩽ r) tmp[k++] = a[i] ⩽ a[j] ? a[i++] : a[j++];
// pick the smaller front
    while (i ⩽ m) tmp[k++] = a[i++];
    while (j ⩽ r) tmp[k++] = a[j++];
    System.arraycopy(tmp, 0, a, l, tmp.length);
}
```

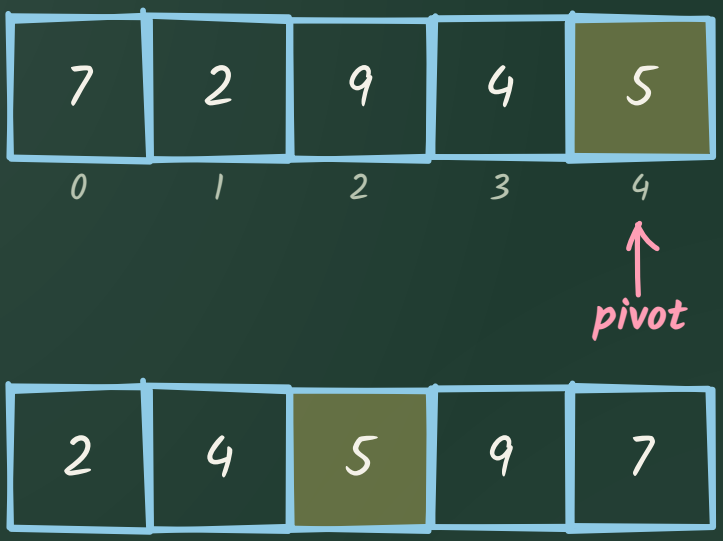
Fun fact

Merge sort is the engine behind Java's `Collections.sort` / `Arrays.sort(Object[])` (TimSort, a merge-sort family).

SORTING

Quick Sort: Pick a Pivot

Pick a ***pivot***; move smaller items to its left and bigger to its right; the pivot is now in its final place. Repeat on both sides. ***Average O(n log n)***, worst O(n²), no extra array.



After partition: [smaller] pivot [bigger].

```
static void quickSort(int[] a, int lo, int hi) {
    if (lo ⩾ hi) return;
    int p = partition(a, lo, hi);
    quickSort(a, lo, p - 1);
    quickSort(a, p + 1, hi);
}
static int partition(int[] a, int lo, int hi) {
    int pivot = a[hi], i = lo;                    // i = next slot for a
small value
    for (int j = lo; j < hi; j++)
        if (a[j] < pivot) { int t = a[i]; a[i] = a[j]; a[j] = t; i++; }
    int t = a[i]; a[i] = a[hi]; a[hi] = t;        // pivot into place
    return i;
}
```

Quickselect: k-th smallest without full sort

Partition once; the pivot's final index tells you which side holds the k-th item. Keep only that side: ***average O(n)***.

Sorting in Java: Comparator & Lambdas

SORTING

In real code you rarely write a sort. You call Java's and tell it **how to compare**.

```
static void demo() {
    int[] a = {5, 2, 9};
    Arrays.sort(a);                                          // primitives:
dual-pivot quicksort
    String[] w = {"pear", "fig", "apple", "kiwi"};
    Arrays.sort(w, (x, y) → x.length() ≠ y.length() ? x.length() -
y.length() : x.compareTo(y));
    System.out.println(Arrays.toString(a) + " " + Arrays.toString(w));
    List<Integer> list = new ArrayList◇(List.of(3, 1, 2));
    Collections.sort(list, Collections.reverseOrder());      // descending
    System.out.println(list);
    int[][] iv = {{5, 8}, {1, 3}, {2, 4}};
    Arrays.sort(iv, (p, q) → Integer.compare(p[0], q[0]));  // sort rows
by first number
    System.out.println(Arrays.deepToString(iv));
}
```

output:
```
[2, 5, 9] [fig, kiwi, pear, apple]
[3, 2, 1]
[[1, 3], [2, 4], [5, 8]]
```

Comparable: the class knows its own order

```
static class Student implements Comparable<Student> {
    String name; int marks;
    Student(String n, int m) { name = n; marks = m; }
    public int compareTo(Student o) { return o.marks - marks; }   // high
marks first
}
```

Trap

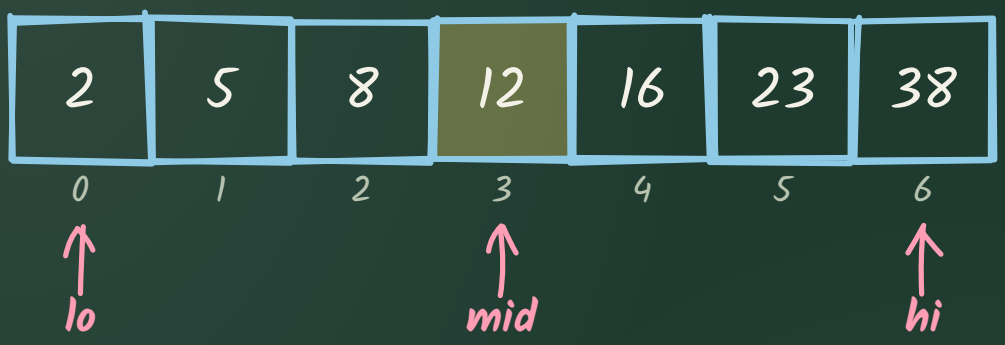
Use `Integer.compare(a, b)` instead of `a - b`: subtraction can **overflow** with huge or negative values.

- `Arrays.sort(int[])` is fast; for `Integer[]` or objects you can pass a **Comparator**.
- Sorting objects is **stable**: equal items keep their original order.

BINARY SEARCH

Binary Search: Halve It

On a **sorted** array, look at the **middle**: too big → search the left half; too small → search the right half. Each step **halves** the range: **O(log n)**. 1 million items need only ~20 steps.



Find 23: mid = 12 is too small → lo moves to the right half.

```
static int binarySearch(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;          // avoids overflow of (lo + hi)
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Classic bug
Always write `mid = lo + (hi - lo) / 2`. `(lo + hi) / 2` can overflow for big arrays.

- Java built-ins: `Arrays.binarySearch(a, x)`, `Collections.binarySearch(list, x)`.
- Needs **sorted** input. Not sorted? Sort first or use a HashSet.

BINARY SEARCH

Binary Search: Variations

First position ≥ target (lower bound)

Instead of stopping when found, keep shrinking to the *leftmost* answer. This one pattern gives first/last occurrence and insert position.

```
static int lowerBound(int[] a, int target) {      // first index with a[i] ≥ target
    int lo = 0, hi = a.length;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < target) lo = mid + 1; else hi = mid;
    }
    return lo;
}
static int[] firstLast(int[] a, int t) {
    int first = lowerBound(a, t);
    if (first == a.length || a[first] ≠ t) return new int[]{-1, -1};
    return new int[]{first, lowerBound(a, t + 1) - 1};
}
```

Search in a rotated sorted array

One half is always properly sorted. Check if the target lies in the sorted half; if not, go to the other.

```
static int searchRotated(int[] a, int t) {
    int lo = 0, hi = a.length - 1;
    while (lo ⩽ hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) return mid;
        if (a[lo] ⩽ a[mid]) {                          // left half sorted
            if (a[lo] ⩽ t && t < a[mid]) hi = mid - 1; else lo = mid + 1;
        } else {                                       // right half sorted
            if (a[mid] < t && t ⩽ a[hi]) lo = mid + 1; else hi = mid - 1;
        }
    }
    return -1;
}
```

BINARY SEARCH

Binary Search on the Answer

Sometimes the array is not sorted, but the **answer range is**: if speed 5 works then 6 also works. Binary search the **answer** itself: *'what is the smallest value that works?'*



Answers go from ✘ to ✔ once. Find the first ✔.

1. Integer square root

```
static int mySqrt(int x) {
    long lo = 0, hi = x;
    while (lo < hi) {
        long mid = (lo + hi + 1) / 2;
        if (mid * mid ⩽ x) lo = mid; else hi = mid - 1;
    }
    return (int) lo;
}
```

2. Real-world: slowest eating speed (Koko bananas)

Piles of bananas, `h` hours. Find the **smallest speed** (bananas/hour) that finishes in time.

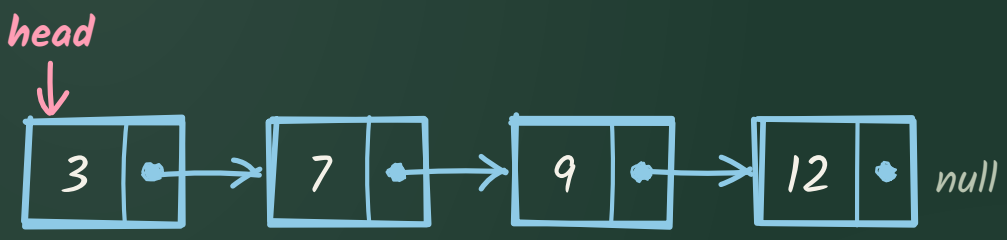
```
static int minSpeed(int[] piles, int h) {
    int lo = 1, hi = 0;
    for (int p : piles) hi = Math.max(hi, p);
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        long hours = 0;
        for (int p : piles) hours += (p + mid - 1) / mid;      // ceil(p / mid)
        if (hours ⩽ h) hi = mid; else lo = mid + 1;
    }
    return lo;
}
```

Spot it
Clues: 'minimum **maximum**', 'smallest speed / capacity / days such that…'. Write a `canDo(x)` check, then binary search.

LINKED LIST

Linked List: Nodes & Pointers

A *linked list* is a chain of *nodes*. Each node holds a value and a *reference* to the next node. No fixed size, no shifting: inserting at the front is *O(1)*. The price: no instant `a[i]`, you must walk the chain.



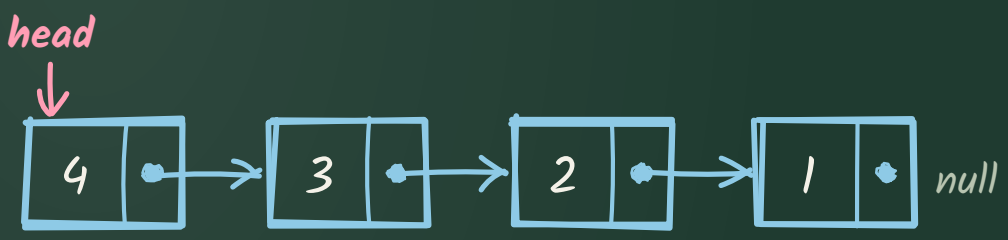
Each box: [value | next arrow]. The last next is null.

```
static class ListNode {
    int val;
    ListNode next;
    ListNode(int v) { val = v; }
}
static ListNode insertHead(ListNode h, int v) {
    ListNode n = new ListNode(v);
    n.next = h;                                  // new node points at old
head
    return n;                                    // and becomes the head
}
static ListNode insertTail(ListNode h, int v) {
    ListNode n = new ListNode(v);
    if (h == null) return n;
    ListNode c = h;
    while (c.next ≠ null) c = c.next;            // walk to the last node
    c.next = n;
    return h;
}
static ListNode delete(ListNode h, int v) {
    ListNode dummy = new ListNode(0);            // dummy head: no special
case
    dummy.next = h;
    ListNode c = dummy;
    while (c.next ≠ null) {
        if (c.next.val == v) { c.next = c.next.next; break; }
        c = c.next;
    }
    return dummy.next;
}
```

LINKED LIST

Linked List: Reverse

The #1 linked-list interview question. Walk the list and **flip each arrow** to point backwards. Keep three variables: `prev`, `cur`, `next`.



After reversing 1→2→3→4 we get 4→3→2→1.

iterative: O(n) time, O(1) space

```
static ListNode reverse(ListNode head) {
    ListNode prev = null, cur = head;
    while (cur ≠ null) {
        ListNode next = cur.next;      // 1. remember the rest
        cur.next = prev;               // 2. flip the arrow
        prev = cur;                    // 3. step forward
        cur = next;
    }
    return prev;                       // new head
}
```

recursive: O(n) stack space

```
static ListNode reverseRec(ListNode head) {
    if (head == null || head.next == null) return head;      // base case
    ListNode newHead = reverseRec(head.next);                // reverse the
rest
    head.next.next = head;                                   // next node
points back
    head.next = null;
    return newHead;
}
```

Common bug

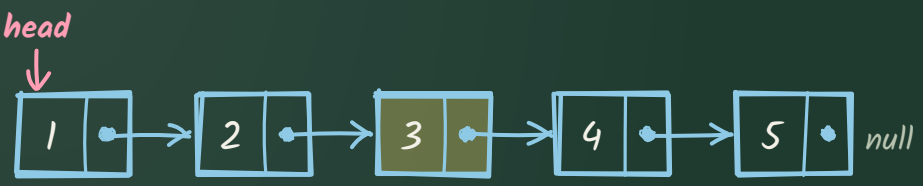
Pointer drill: before changing `cur.next`, **save** it first, otherwise you lose the rest of the list!

- Draw 3 nodes on paper and move `prev / cur / next` by hand.
- Variation: reverse only part of the list (between positions m and n).

LINKED LIST

Fast & Slow Pointers

Two pointers on one list: ***slow*** moves 1 step, ***fast*** moves 2. When fast reaches the end, slow is at the ***middle***. If there is a ***cycle***, fast eventually catches slow (***Floyd's algorithm***).



Slow ends in the middle (3).

```
static ListNode middle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast ≠ null && fast.next ≠ null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
static boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast ≠ null && fast.next ≠ null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;       // they met inside a loop
    }
    return false;
}
```

After they meet, restart one pointer at `head`; move both 1 step: they meet at the cycle start.

```
static ListNode cycleStart(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast ≠ null && fast.next ≠ null) {
        slow = slow.next; fast = fast.next.next;
        if (slow == fast) {
            slow = head;
            while (slow ≠ fast) { slow = slow.next; fast = fast.next; }
            return slow;
        }
    }
    return null;
}
```

Linked List: Merge, Remove & Add

LINKED LIST

1. Merge two sorted lists

```
static ListNode merge(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(0), tail = dummy;
    while (a ≠ null && b ≠ null) {
        if (a.val ⩽ b.val) { tail.next = a; a = a.next; }
        else               { tail.next = b; b = b.next; }
        tail = tail.next;
    }
    tail.next = (a ≠ null) ? a : b;              // attach the leftover
    return dummy.next;
}
```

2. Remove the n-th node from the end

```
static ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode fast = dummy, slow = dummy;
    for (int i = 0; i < n; i++) fast = fast.next;
    while (fast.next ≠ null) { fast = fast.next; slow = slow.next; }
    slow.next = slow.next.next;
    return dummy.next;
}
```

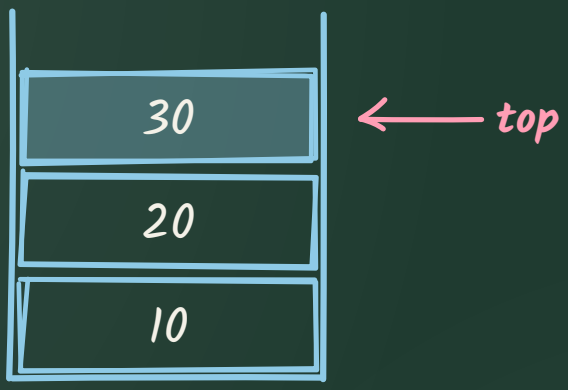
3. Add two numbers stored in reverse (digits)

```
static ListNode addTwo(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(0), t = dummy;
    int carry = 0;
    while (a ≠ null || b ≠ null || carry > 0) {
        int sum = carry + (a ≠ null ? a.val : 0) + (b ≠ null ? b.val :
0);
        t.next = new ListNode(sum % 10);
        carry = sum / 10;
        t = t.next;
        if (a ≠ null) a = a.next;
        if (b ≠ null) b = b.next;
    }
    return dummy.next;
}
```

STACK

Stack: Last In, First Out

A *stack* is like a pile of plates: you add and remove only at the *top*. *LIFO* = Last In, First Out. Push, pop and peek are all *O(1)*.



push(10), push(20), push(30) → pop() gives 30 first.

```
static void demo() {
    Deque<Integer> st = new ArrayDeque<>();
    st.push(10); st.push(20); st.push(30);
    System.out.println(st.peek() + " " + st.pop() + " " + st.pop() + "
size=" + st.size());
    System.out.println(st.isEmpty());
}
static String reverse(String s) {                    // stack reverses order
    Deque<Character> st = new ArrayDeque<>();
    for (char c : s.toCharArray()) st.push(c);
    StringBuilder sb = new StringBuilder();
    while (!st.isEmpty()) sb.append(st.pop());
    return sb.toString();
}
```

```
output: 30 30 20 size=1
false
```

Where stacks hide in real life

- *Undo / redo* in editors, *browser back button*.
- *Function calls* (the call stack) and *recursion*.
- *Matching brackets* in compilers, *expression evaluation*.

Java tip

In Java use `Deque<Integer> st = new ArrayDeque<>()` with `push / pop / peek`. The old `Stack` class is slow and outdated.

Stack Problems: Brackets, MinStack, RPN

STACK

1. Valid parentheses

Push every opener. On a closer, the top of the stack must be its matching opener.

```
static boolean isValid(String s) {
    Deque<Character> st = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.isEmpty()) return false;
            char o = st.pop();
            if ((c == ')' && o != '(') || (c == ']' && o != '[') || (c ==
'}' && o != '{')) return false;
        }
    }
    return st.isEmpty();
}
```

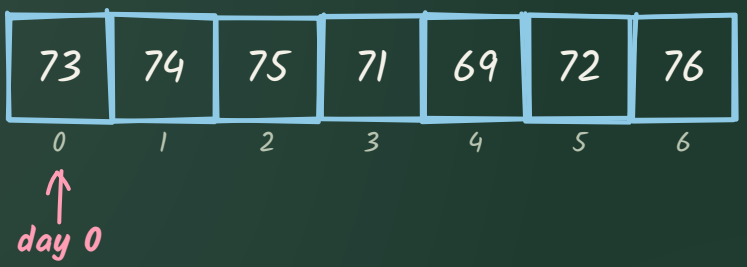
2. Min stack: getMin() in O(1)

```
static class MinStack {
    Deque<Integer> st = new ArrayDeque<>(), mins = new ArrayDeque<>();
    void push(int x) {
        st.push(x);
        if (mins.isEmpty() || x <= mins.peek()) mins.push(x);   // remember
each new minimum
    }
    void pop() { if (st.pop().equals(mins.peek())) mins.pop(); }
    int top() { return st.peek(); }
    int getMin() { return mins.peek(); }
}
```

STACK

Monotonic Stack: Next Greater

Keep the stack in ***sorted order*** (e.g. decreasing). When a new number breaks the order, pop the smaller ones: ***they just found their answer***. Each item enters and leaves once: ***O(n)***.



Daily temperatures: how many days until a warmer day?

```java
static int[] nextGreater(int[] a) {
    int[] res = new int[a.length];
    Arrays.fill(res, -1);
    Deque<Integer> st = new ArrayDeque<>();          // stores indexes
    for (int i = 0; i < a.length; i++) {
        while (!st.isEmpty() && a[st.peek()] < a[i])
            res[st.pop()] = a[i];                    // a[i] is the next
greater for them
        st.push(i);
    }
    return res;
}
static int[] dailyTemperatures(int[] t) {
    int[] res = new int[t.length];
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < t.length; i++) {
        while (!st.isEmpty() && t[st.peek()] < t[i]) {
            int j = st.pop();
            res[j] = i - j;                          // days waited
        }
        st.push(i);
    }
    return res;
}
```

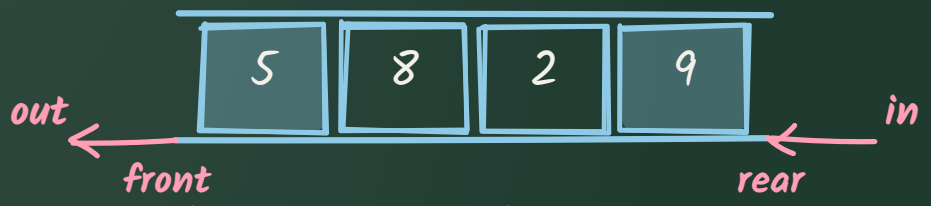
Clue

Spot it: 'next greater / smaller element', 'days until warmer', 'stock span', 'largest rectangle in histogram'.

QUEUE

Queue & Deque: First In, First Out

A *queue* is a line at a ticket counter: join at the *rear*, leave from the *front*. *FIFO*. `offer` and `poll` are *O(1)*. A *deque* works at both ends.



offer(x) adds at the rear; poll() removes from the front.

```
static void demo() {
    Queue<Integer> q = new ArrayDeque<>();
    q.offer(5); q.offer(8); q.offer(2);
    System.out.println(q.peek() + " " + q.poll() + " " + q.poll() + " " +
q.size());
    Deque<Integer> dq = new ArrayDeque<>();
    dq.addFirst(1); dq.addLast(2); dq.addFirst(0);
    System.out.println(dq + " first=" + dq.peekFirst() + " last=" +
dq.peekLast());
}
```

output:
```
5 5 8 1
[0, 1, 2] first=0 last=2
```

Queue using two stacks (classic!)

```
static class MyQueue {
    Deque<Integer> in = new ArrayDeque<>(), out = new ArrayDeque<>();
    void push(int x) { in.push(x); }
    int pop() { move(); return out.pop(); }
    int peek() { move(); return out.peek(); }
    boolean empty() { return in.isEmpty() && out.isEmpty(); }
    private void move() { if (out.isEmpty()) while (!in.isEmpty())
out.push(in.pop()); }   // reverses order
}
```

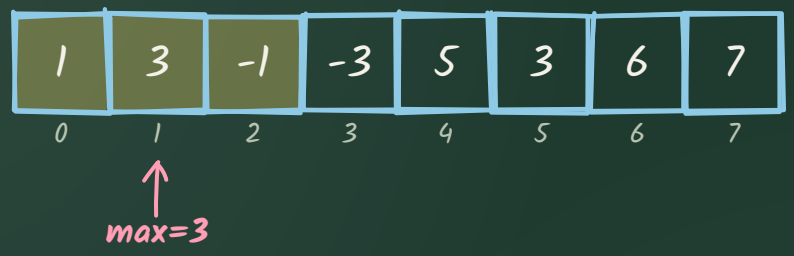
Use cases

Real life: print jobs, customer support tickets, *BFS* on graphs and trees.

Deque Problems: Sliding Window Maximum

QUEUE

Brute force checks k numbers per window: O(n·k). A **monotonic deque** keeps indexes of candidates in **decreasing value order**, so the front is always the maximum: **O(n)**.



Window size 3 → maxima 3, 3, 5, 5, 6, 7

```
static int[] slidingMax(int[] a, int k) {
    int[] res = new int[a.length - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();              // indexes, values
decreasing
    for (int i = 0; i < a.length; i++) {
        if (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();
// left the window
        while (!dq.isEmpty() && a[dq.peekLast()] < a[i]) dq.pollLast();
// smaller ones are useless
        dq.offerLast(i);
        if (i >= k - 1) res[i - k + 1] = a[dq.peekFirst()];
    }
    return res;
}
```

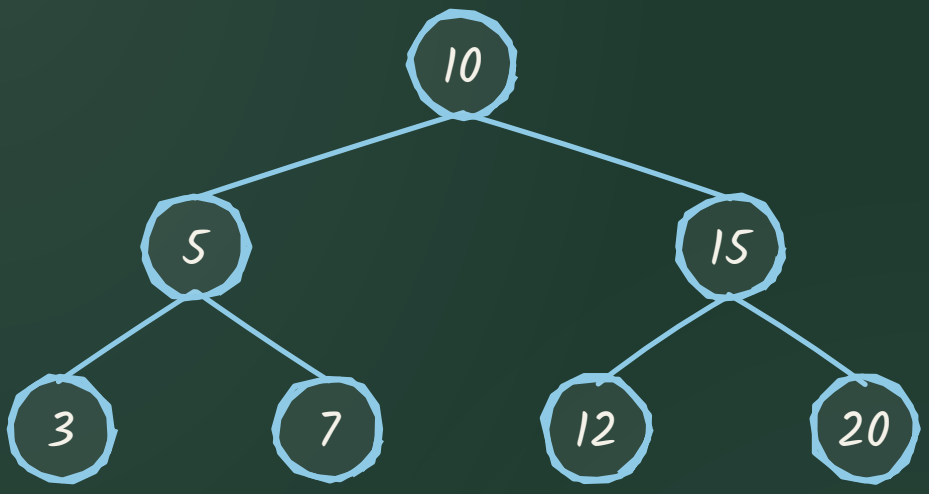
Real-world: first non-repeating character in a stream

```
static class Stream {
    Queue<Character> q = new ArrayDeque<>();
    int[] cnt = new int[26];
    void add(char c) { cnt[c - 'a']++; q.offer(c); }
    char firstUnique() {
        while (!q.isEmpty() && cnt[q.peek() - 'a'] > 1) q.poll();   // drop
repeats from the front
        return q.isEmpty() ? '#' : q.peek();
    }
}
```

TREES

Trees: Nodes, Roots & Leaves

A *tree* is a hierarchy: one *root*, and each *node* has *children*. Think of a company chart or folders on your computer. In a *binary tree* every node has at most *2 children*: left and right.



root = 10, leaves = 3, 7, 12, 20. Height = 2 (edges on the longest path).

```
static class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int v) { val = v; }
}
static int height(TreeNode n) {                    // -1 for empty, so a
leaf has height 0
    if (n == null) return -1;
    return 1 + Math.max(height(n.left), height(n.right));
}
static int size(TreeNode n) { return n == null ? 0 : 1 + size(n.left) +
size(n.right); }
static int leaves(TreeNode n) {
    if (n == null) return 0;
    if (n.left == null && n.right == null) return 1;
    return leaves(n.left) + leaves(n.right);
}
```

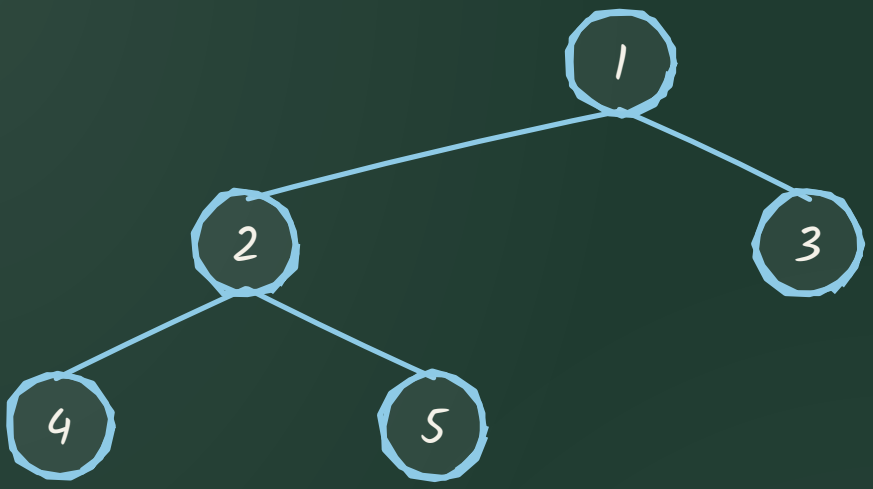
Core idea

Almost every tree problem is *recursion*: solve for the left subtree, solve for the right, then combine at the current node.

TREES

Tree Traversals: DFS Orders

A *traversal* visits every node. The three DFS orders differ in *when* you handle the current node.



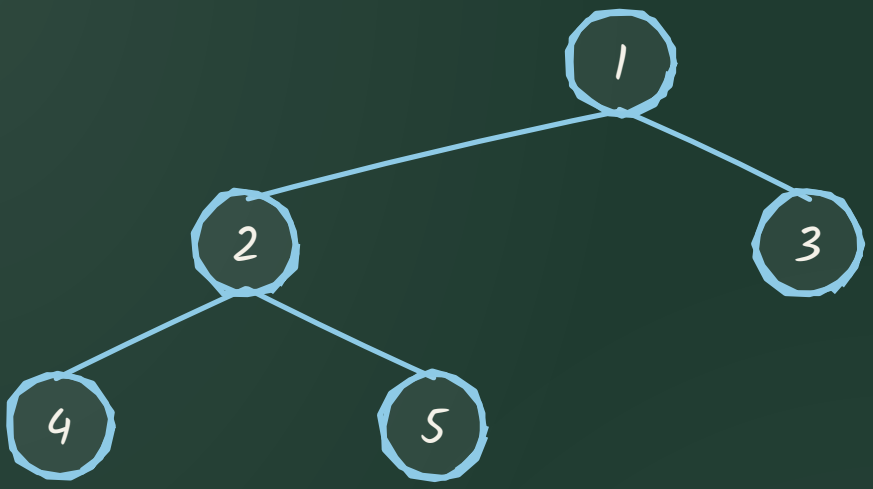
Preorder 1 2 4 5 3 · Inorder 4 2 5 1 3 · Postorder 4 5 2 3 1

```
static void inorder(TreeNode n, List<Integer> out) {
    if (n == null) return;
    inorder(n.left, out);
    out.add(n.val);                // move this line first = preorder,
last = postorder
    inorder(n.right, out);
}
static List<Integer> inorderIter(TreeNode root) {       // same result
without recursion
    List<Integer> out = new ArrayList<>();
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root;
    while (cur != null || !st.isEmpty()) {
        while (cur != null) { st.push(cur); cur = cur.left; }   // go as
far left as possible
        cur = st.pop();
        out.add(cur.val);
        cur = cur.right;
    }
    return out;
}
```

TREES

Level Order (BFS) & Views

BFS visits a tree **level by level** using a **queue**. Process exactly `size` nodes per level.



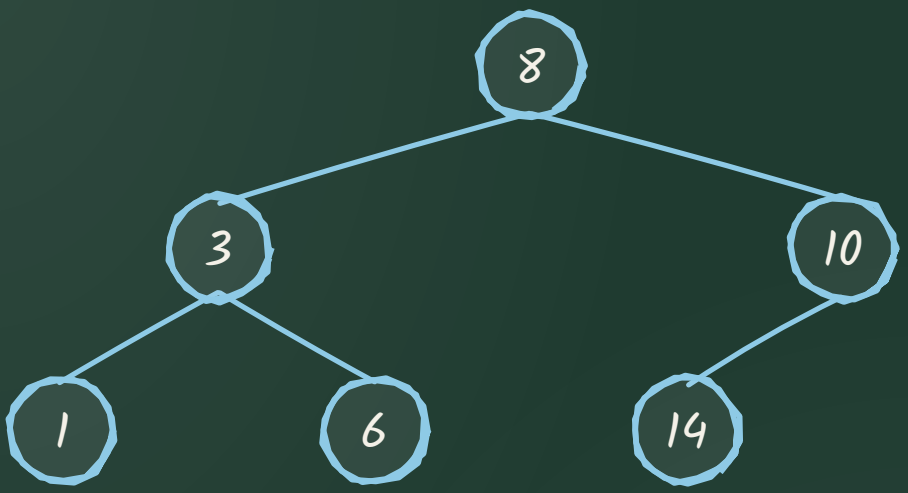
Level order: [1] [2,3] [4,5]

```
static List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();                    // nodes in this level
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode n = q.poll();
            level.add(n.val);
            if (n.left != null) q.offer(n.left);
            if (n.right != null) q.offer(n.right);
        }
        res.add(level);
    }
    return res;
}
```

TREES

Binary Search Tree (BST)

A **BST** keeps order: **left subtree < node < right subtree**, for every node. So search, insert and delete take **O(height)**, which is **O(log n)** when balanced.



Searching 6: 8 → left to 3 → right to 6. Only 3 steps.

```
static TreeNode insert(TreeNode n, int v) {
    if (n == null) return new TreeNode(v);
    if (v < n.val) n.left = insert(n.left, v);
    else if (v > n.val) n.right = insert(n.right, v);
    return n;
}
static boolean search(TreeNode n, int v) {
    if (n == null) return false;
    if (v == n.val) return true;
    return v < n.val ? search(n.left, v) : search(n.right, v);
}
static boolean isValidBST(TreeNode n, long lo, long hi) {      // every
node must fit a (lo, hi) range
    if (n == null) return true;
    if (n.val ⩽ lo || n.val ⩾ hi) return false;
    return isValidBST(n.left, lo, n.val) && isValidBST(n.right, n.val, hi);
}
```

Common bug

Validating a BST by comparing only with the **parent** is a famous mistake. Pass down the allowed **range**.

Tree Problems: Depth, Diameter, LCA

TREES

1. Maximum depth & invert tree

```
static int maxDepth(TreeNode n) { return n == null ? 0 : 1 +
Math.max(maxDepth(n.left), maxDepth(n.right)); }
static TreeNode invert(TreeNode n) {                    // mirror the tree
    if (n == null) return null;
    TreeNode t = n.left; n.left = invert(n.right); n.right = invert(t);
    return n;
}
```

2. Diameter: longest path between any two nodes

At each node the best path through it = *left height + right height*. Track the maximum while computing heights.

```
static int best;
static int depth(TreeNode n) {
    if (n == null) return 0;
    int l = depth(n.left), r = depth(n.right);
    best = Math.max(best, l + r);              // path through this node
    return 1 + Math.max(l, r);
}
static int diameter(TreeNode root) { best = 0; depth(root); return best; }
```

3. Lowest common ancestor (LCA)

```
static TreeNode lca(TreeNode n, TreeNode p, TreeNode q) {
    if (n == null || n == p || n == q) return n;
    TreeNode l = lca(n.left, p, q), r = lca(n.right, p, q);
    return (l ≠ null && r ≠ null) ? n : (l ≠ null ? l : r);   // p and q
on both sides → here
}
```

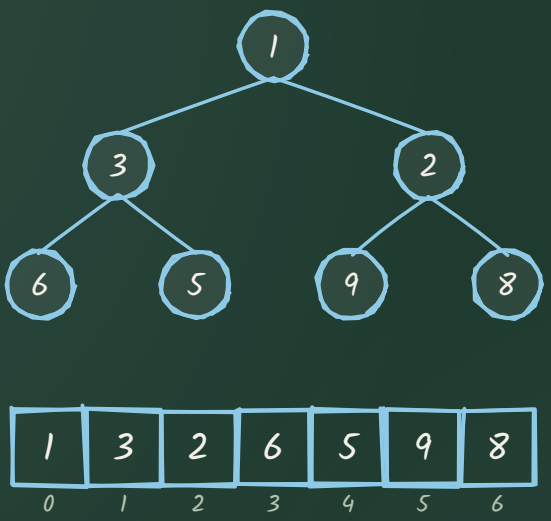
4. Path sum

```
static boolean hasPathSum(TreeNode n, int target) {
    if (n == null) return false;
    if (n.left == null && n.right == null) return target == n.val;
    return hasPathSum(n.left, target - n.val) || hasPathSum(n.right, target
- n.val);
}
```

HEAP

Heap & PriorityQueue

A **heap** is a tree stored in an array where every parent is **smaller** than its children (**min-heap**). The smallest item is always on **top**: peek is **O(1)**, add and remove are **O(log n)**. Java's heap is `PriorityQueue`.



Min-heap as a tree and as an array. Children of i are 2i+1 and 2i+2.

```
static void demo() {
    PriorityQueue<Integer> min = new PriorityQueue<>();
// smallest first
    min.add(5); min.add(1); min.add(3);
    System.out.println(min.peek() + " " + min.poll() + " " + min.poll());
    PriorityQueue<Integer> max = new PriorityQueue<>
(Collections.reverseOrder());    // largest first
    max.addAll(List.of(5, 1, 3));
    System.out.println(max.poll());
    PriorityQueue<int[]> byDist = new PriorityQueue<>((a, b) -> a[1] -
b[1]);        // custom order
    byDist.add(new int[]{7, 30}); byDist.add(new int[]{8, 10});
    System.out.println(byDist.poll()[0]);
}
```

```
output: 1 1 3
5
8
```

Careful

Printing a PriorityQueue does **not** show sorted order, only `poll()` does. Use a heap when you repeatedly need the **smallest / largest** item.

HEAP

Heap Problems: Top K

1. K-th largest element

Keep a *min-heap of size k*. If it grows beyond k, remove the smallest. The top is the answer. *O(n log k)*.

```
static int kthLargest(int[] a, int k) {
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    for (int x : a) {
        pq.add(x);
        if (pq.size() > k) pq.poll();        // drop the smallest
    }
    return pq.peek();
}
```

2. Top K frequent elements

```
static List<Integer> topKFrequent(int[] a, int k) {
    Map<Integer, Integer> cnt = new HashMap<>();
    for (int x : a) cnt.merge(x, 1, Integer::sum);
    PriorityQueue<Integer> pq = new PriorityQueue<>((x, y) -> cnt.get(x) -
cnt.get(y));   // least frequent on top
    for (int key : cnt.keySet()) {
        pq.add(key);
        if (pq.size() > k) pq.poll();
    }
    return new ArrayList<>(pq);
}
```

3. K closest points to the origin

```
static int[][] kClosest(int[][] pts, int k) {
    PriorityQueue<int[]> pq = new PriorityQueue<>((p, q) -> (q[0]*q[0] +
q[1]*q[1]) - (p[0]*p[0] + p[1]*p[1]));  // farthest on top
    for (int[] p : pts) {
        pq.add(p);
        if (pq.size() > k) pq.poll();
    }
    return pq.toArray(new int[0][]);
}
```

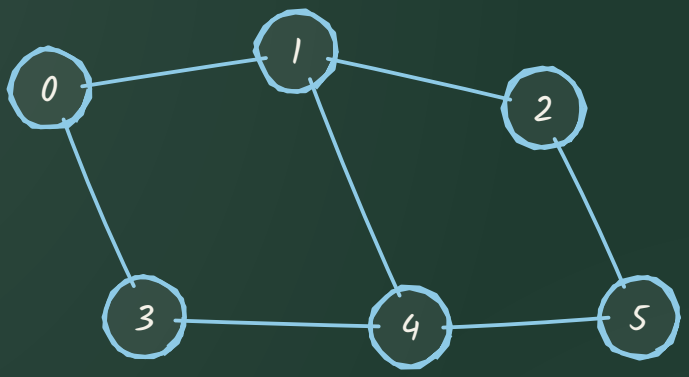
Spot it

Pattern: *'top K', 'K-th', 'K closest'* → a heap of size *K*. For the K *largest* use a *min*-heap (and vice versa).

GRAPHS

Graphs: The Basics

A ***graph*** = ***nodes (vertices)*** connected by ***edges***. It models maps, friends, web links, flights. Edges can be ***undirected*** (two-way) or ***directed*** (one-way) and may carry ***weights*** (distance).



6 nodes, 7 undirected edges

	Adjacency list	Matrix
Space	O(V+E)	O(V²)
Is edge (u,v)?	O(degree)	O(1)
Best for	sparse graphs (most)	dense graphs

```
static List<List<Integer>> build(int n, int[][] edges) {
    List<List<Integer>> g = new ArrayList<>();
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    for (int[] e : edges) {
        g.get(e[0]).add(e[1]);
        g.get(e[1]).add(e[0]);              // skip this line for a
directed graph
    }
    return g;
}
```

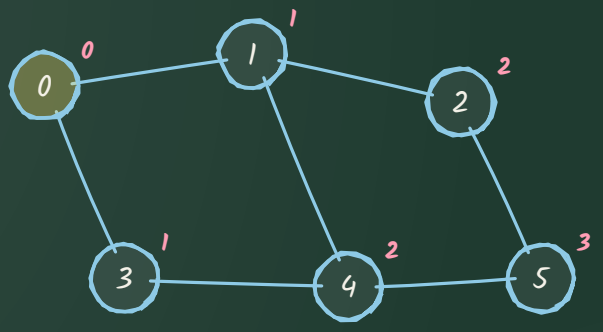
Step 1

Interview inputs: edge lists `int[][] edges`, a grid, or `n` nodes numbered 0..n-1. Convert to an adjacency list first.

GRAPHS

BFS: Breadth-First Search

BFS explores **level by level** like ripples in a pond, using a **queue**. On an unweighted graph BFS finds the **shortest path** (fewest edges). Time **O(V + E)**.



Red numbers = distance from node 0.

```
static List<Integer> bfs(List<List<Integer>> g, int start) {
    boolean[] seen = new boolean[g.size()];
    Queue<Integer> q = new ArrayDeque<>();
    List<Integer> order = new ArrayList<>();
    seen[start] = true; q.offer(start);                // mark when you
PUSH
    while (!q.isEmpty()) {
        int u = q.poll();
        order.add(u);
        for (int v : g.get(u))
            if (!seen[v]) { seen[v] = true; q.offer(v); }
    }
    return order;
}
static int shortest(List<List<Integer>> g, int s, int t) {
    int[] dist = new int[g.size()];
    Arrays.fill(dist, -1);
    Queue<Integer> q = new ArrayDeque<>();
    dist[s] = 0; q.offer(s);
    while (!q.isEmpty()) {
        int u = q.poll();
        if (u == t) return dist[u];
        for (int v : g.get(u)) if (dist[v] < 0) { dist[v] = dist[u] + 1;
q.offer(v); }
    }
    return -1;
}
```

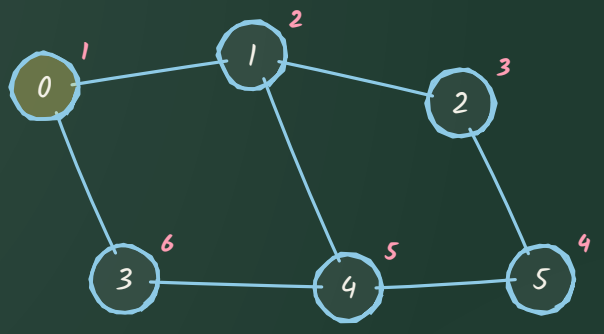
Common bug

Mark visited when you push, not when you pop. Otherwise a node enters the queue many times.

GRAPHS

DFS: Depth-First Search

DFS goes **as deep as possible**, then backtracks. Use recursion (the call stack) or your own stack. Perfect for exploring everything, counting **connected components** and detecting cycles.



One DFS order from 0: 0 → 1 → 2 → 5 → 4 → 3

```
static void dfs(List<List<Integer>> g, int u, boolean[] seen, List<Integer>
order) {
    seen[u] = true;
    order.add(u);
    for (int v : g.get(u))
        if (!seen[v]) dfs(g, v, seen, order);
}
static int components(List<List<Integer>> g) {
    boolean[] seen = new boolean[g.size()];
    int count = 0;
    for (int u = 0; u < g.size(); u++)
        if (!seen[u]) { dfs(g, u, seen, new ArrayList<>()); count++; }
    return count;
}
```

Choose

BFS vs DFS: need the *shortest* path → BFS. Need to *explore / count / search everything* → DFS (shorter code).

GRAPHS

Graphs on a Grid: Islands

A 2D grid is a hidden graph: each cell is a node; its **4 neighbours** (up, down, left, right) are the edges. Many interview problems are grid DFS/BFS.



'1' = land, '0' = water → 2 islands

```
static int numIslands(char[][] g) {
    int count = 0;
    for (int r = 0; r < g.length; r++)
        for (int c = 0; c < g[0].length; c++)
            if (g[r][c] == '1') { count++; sink(g, r, c); }
    return count;
}
static void sink(char[][] g, int r, int c) {
    if (r < 0 || c < 0 || r ≥ g.length || c ≥ g[0].length || g[r][c] ≠
'1') return;
    g[r][c] = '0';                                    // mark visited
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};       // direction
array
    for (int[] d : dirs) sink(g, r + d[0], c + d[1]);
}
```

Same trick

Other grid problems: **rotten oranges** (multi-source BFS), **shortest path in a maze** (BFS), **flood fill**, **surrounded regions**.

GRAPHS

Dijkstra & Topological Sort

Dijkstra: shortest path with weights

Always expand the ***closest unfinished node*** with a min-heap. Needs ***non-negative*** weights. ***O((V+E) log V)***.

```
static int[] dijkstra(int n, int[][] edges, int src) {            // edges: {u, v, weight}
    List<List<int[]>> g = new ArrayList<>();
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    for (int[] e : edges) { g.get(e[0]).add(new int[]{e[1], e[2]});
g.get(e[1]).add(new int[]{e[0], e[2]}); }
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);  dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
// {node, distance}
    pq.add(new int[]{src, 0});
    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        if (cur[1] > dist[cur[0]]) continue;                       // stale entry
        for (int[] nb : g.get(cur[0])) {
            int nd = cur[1] + nb[1];
            if (nd < dist[nb[0]]) { dist[nb[0]] = nd; pq.add(new int[]
{nb[0], nd}); }
        }
    }
    return dist;
}
```

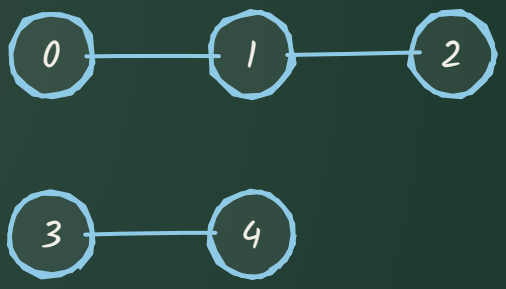
topological sort (course prerequisites)

```
static List<Integer> topo(int n, int[][] edges) {          // edge {a, b}: a must come before b
    List<List<Integer>> g = new ArrayList<>();
    int[] indeg = new int[n];
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    for (int[] e : edges) { g.get(e[0]).add(e[1]); indeg[e[1]]++; }
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.offer(i);
    List<Integer> out = new ArrayList<>();
    while (!q.isEmpty()) {
        int u = q.poll(); out.add(u);
        for (int v : g.get(u)) if (--indeg[v] == 0) q.offer(v);
    }
    return out.size() == n ? out : new ArrayList<>();      // empty list = cycle
}
```

GRAPHS

Union-Find (Disjoint Set)

Union-Find tracks **groups**. `find(x)`: which group is x in? `union(a, b)`: merge two groups. With two tricks it is almost **O(1)** per operation.



Groups: {0,1,2} and {3,4}. Are 2 and 4 connected? Compare group leaders.

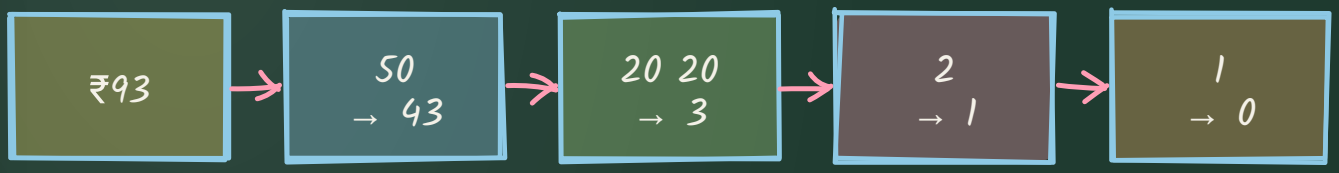
```
static class DSU {
    int[] parent, size;
    DSU(int n) {
        parent = new int[n]; size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
    }
    int find(int x) {
        while (parent[x] ≠ x) { parent[x] = parent[parent[x]]; x =
parent[x]; }   // path compression
        return x;
    }
    boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;                    // already
together
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra; size[ra] += size[rb];         // small under
big
        return true;
    }
}
```

- **Cycle in an undirected graph:** if `union` returns false for an edge, that edge forms a cycle.
- **Count components:** start with n, subtract 1 per successful union.
- Also: Kruskal's MST, number of provinces, accounts merge.

GREEDY

Greedy Algorithms

A *greedy* algorithm takes the *best-looking choice right now*, never looks back, and hopes this gives the best overall answer. Fast and simple, but only correct when the problem allows it, so always *justify* the choice.



Making ₹93 with coins 50,20,10,5,2,1: biggest coin first = 5 coins.

```
static int minCoins(int amount, int[] coinsDesc) {        // coins sorted
high → low
    int count = 0;
    for (int c : coinsDesc) { count += amount / c; amount %= c; }
    return count;
}
static boolean canJump(int[] a) {                          // can we reach
the last index?
    int reach = 0;
    for (int i = 0; i < a.length; i++) {
        if (i > reach) return false;                       // stuck before i
        reach = Math.max(reach, i + a[i]);
    }
    return true;
}
```

Warning

Greedy can fail! Coins {1,3,4}, amount 6: greedy gives 4+1+1 = 3 coins, but 3+3 = ***2 coins***. Then you need ***DP***.

- Clues: ***maximum, minimum, earliest, fewest***. Sorting first usually helps.
- Test greedy on a tiny counter-example before trusting it.

GREEDY

Greedy: Intervals & Stocks

1. Most meetings in one room: finish earliest first

```
static int maxMeetings(int[][] meetings) {
    Arrays.sort(meetings, (a, b) → Integer.compare(a[1], b[1]));    // by
end time
    int end = Integer.MIN_VALUE, count = 0;
    for (int[] m : meetings)
        if (m[0] ⩾ end) { count++; end = m[1]; }
    return count;
}
```

2. Merge overlapping intervals

```
static int[][] merge(int[][] iv) {
    Arrays.sort(iv, (a, b) → Integer.compare(a[0], b[0]));
    List<int[]> out = new ArrayList◇();
    for (int[] cur : iv) {
        if (out.isEmpty() || out.get(out.size() - 1)[1] < cur[0])
out.add(cur);   // gap: new interval
        else out.get(out.size() - 1)[1] = Math.max(out.get(out.size() - 1)
[1], cur[1]);
    }
    return out.toArray(new int[0][]);
}
```

3. Best time to buy and sell a stock once

```
static int maxProfit(int[] prices) {
    int low = Integer.MAX_VALUE, best = 0;
    for (int p : prices) {
        low = Math.min(low, p);              // cheapest day so far
        best = Math.max(best, p - low);      // sell today?
    }
    return best;
}
```

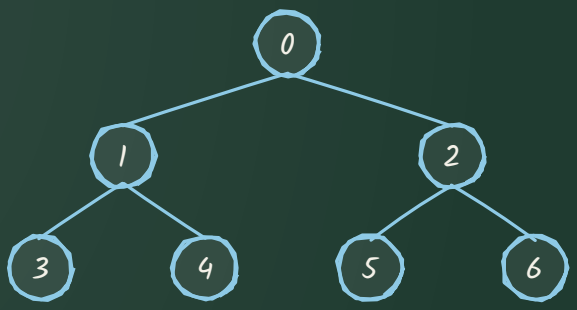
Pattern

Interval problems: ***sort first*** (by start for merging, by end for scheduling).

BACKTRACKING

Backtracking: Try, Undo, Retry

Backtracking builds an answer one choice at a time. Dead end? **Undo** the last choice and try the next, like exploring a maze.



A decision tree: every node is a choice; a dead end sends us back up.

The 3-step template: choose, explore, un-choose

```
static List<List<Integer>> subsets(int[] a) {
    List<List<Integer>> out = new ArrayList<>();
    go(a, 0, new ArrayList<>(), out);
    return out;
}
static void go(int[] a, int i, List<Integer> path, List<List<Integer>> out)
{
    if (i == a.length) { out.add(new ArrayList<>(path)); return; }   //
COPY the path!
    path.add(a[i]);                          // choose: take a[i]
    go(a, i + 1, path, out);                 // explore
    path.remove(path.size() - 1);            // un-choose: undo
    go(a, i + 1, path, out);                 // explore the "skip" branch
}
```

Classic bug

Always add a **copy** `new ArrayList<>(path)`. Adding `path` itself stores a reference that changes later.

- **Prune:** stop early when a choice can never work. That makes it fast.
- Good for: **subsets, permutations, combinations, N-Queens, Sudoku, word search.**

BACKTRACKING

Permutations, Combinations & N-Queens

1. All permutations

```
static List<List<Integer>> permutations(int[] a) {
    List<List<Integer>> out = new ArrayList<>();
    go(a, new boolean[a.length], new ArrayList<>(), out);
    return out;
}
static void go(int[] a, boolean[] used, List<Integer> path,
List<List<Integer>> out) {
    if (path.size() == a.length) { out.add(new ArrayList<>(path)); return;
}
    for (int i = 0; i < a.length; i++) {
        if (used[i]) continue;
        used[i] = true; path.add(a[i]);          // choose
        go(a, used, path, out);                  // explore
        path.remove(path.size() - 1); used[i] = false;   // undo
    }
}
```

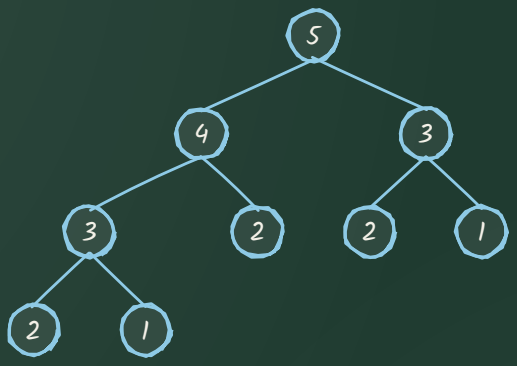
2. N-Queens: count the placements

```
static int nQueens(int n) { return place(0, n, new boolean[n], new
boolean[2 * n], new boolean[2 * n]); }
static int place(int r, int n, boolean[] cols, boolean[] d1, boolean[] d2)
{
    if (r == n) return 1;
    int count = 0;
    for (int c = 0; c < n; c++) {
        int a = r + c, b = r - c + n;                    // the two
diagonals
        if (cols[c] || d1[a] || d2[b]) continue;
        cols[c] = d1[a] = d2[b] = true;
        count += place(r + 1, n, cols, d1, d2);
        cols[c] = d1[a] = d2[b] = false;
    }
    return count;
}
```

DYNAMIC PROGRAMMING

Dynamic Programming: The Idea

DP = recursion + **memory**. If a problem splits into **overlapping** smaller problems, solve each **once**, store the answer, and reuse it.



fib(3) and fib(2) are recomputed again and again: cost ≈ 2^n.

```java
static long fibMemo(int n, long[] memo) {                    // top-down: recursion + cache
    if (n < 2) return n;
    if (memo[n] != 0) return memo[n];
    return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}
static long fibTab(int n) {                                  // bottom-up: loop + table
    if (n < 2) return n;
    long a = 0, b = 1;
    for (int i = 2; i <= n; i++) { long c = a + b; a = b; b = c; }
    return b;
}
```

5 steps to solve any DP

1. Define the **state** : what does `dp[i]` mean?
2. Find the **recurrence** : dp[i] from smaller ones.
3. Set the **base cases** .
4. Choose the **order** (small to big).
5. Return the **answer cell** .

Spot DP

Clues: *count the ways, min/max cost, can we reach, best choice at each step*.

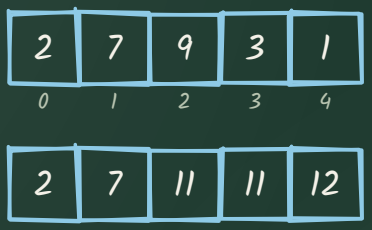
DYNAMIC PROGRAMMING

DP on a Line (1D)

1. Climbing stairs: ways to reach step n (1 or 2 steps)

```
static int climb(int n) {
    int a = 1, b = 1;                       // ways for steps 0 and 1
    for (int i = 2; i ⩽ n; i++) { int c = a + b; a = b; b = c; }
    return b;
}
```

2. House robber: can't rob neighbours



Top: money. Bottom: best total so far → 12 (2 + 9 + 1).

```
static int rob(int[] a) {
    int prev = 0, cur = 0;
    for (int x : a) {
        int next = Math.max(cur, prev + x);       // skip this house, or
rob it
        prev = cur; cur = next;
    }
    return cur;
}
```

3. Coin change: fewest coins

```
static int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);                    // "infinity"
    dp[0] = 0;
    for (int x = 1; x ⩽ amount; x++)
        for (int c : coins)
            if (c ⩽ x) dp[x] = Math.min(dp[x], dp[x - c] + 1);
    return dp[amount] > amount ? -1 : dp[amount];
}
```

DYNAMIC PROGRAMMING

DP on Grids & Strings (2D)

1. Unique paths (move right or down)



Each cell = from above + from left. Answer = 10.

```
static int uniquePaths(int rows, int cols) {
    int[][] dp = new int[rows][cols];
    for (int[] row : dp) Arrays.fill(row, 1);
    for (int i = 1; i < rows; i++)
        for (int j = 1; j < cols; j++) dp[i][j] = dp[i - 1][j] + dp[i][j -
1];
    return dp[rows - 1][cols - 1];
}
```

2. Longest common subsequence

```
static int lcs(String a, String b) {
    int[][] dp = new int[a.length() + 1][b.length() + 1];
    for (int i = 1; i ⩽ a.length(); i++)
        for (int j = 1; j ⩽ b.length(); j++)
            dp[i][j] = a.charAt(i - 1) == b.charAt(j - 1)
                    ? dp[i - 1][j - 1] + 1
                    : Math.max(dp[i - 1][j], dp[i][j - 1]);
    return dp[a.length()][b.length()];
}
```

3. 0/1 knapsack (each item once)

```
static int knapsack(int[] wt, int[] val, int cap) {
    int[] dp = new int[cap + 1];
    for (int i = 0; i < wt.length; i++)
        for (int c = cap; c ⩾ wt[i]; c--)            // backwards: use
each item once
            dp[c] = Math.max(dp[c], dp[c - wt[i]] + val[i]);
    return dp[cap];
}
```

BITS

Bit Manipulation

Numbers are stored in *binary*. Bit tricks are tiny, fast operations that solve neat interview problems.

Operator	Meaning	Example
&	AND	5 & 3 = 1
|	OR	5 | 3 = 7
^	XOR (differ)	5 ^ 3 = 6
<< >>	shift ×2 / ÷2	3 << 2 = 12

```
static void demo() {
    System.out.println((5 & 3) + " " + (5 | 3) + " " + (5 ^ 3) + " " + (3
<< 2) + " " + (12 >> 2));
    System.out.println((-8 >> 1) + " " + (-8 >>> 28));     // >> keeps
sign, >>> fills with zeros
    System.out.println(Integer.toBinaryString(10) + " " +
Integer.bitCount(7));
}
```

output:
```
1 7 6 12 3
-4 15
1010 3
```

```
static boolean isPowerOfTwo(int n) { return n > 0 && (n & (n - 1)) == 0; }
// n & (n-1) clears the lowest 1-bit
static int singleNumber(int[] a) {                  // every number twice
except one: XOR cancels pairs
    int r = 0;
    for (int x : a) r ^= x;
    return r;
}
static int countBits(int n) {
    int c = 0;
    while (n != 0) { n &= n - 1; c++; }             // one set bit removed
per loop
    return c;
}
static boolean isOdd(int n) { return (n & 1) == 1; }
```

Remember

`x ^ x = 0` and `x ^ 0 = x`. That is why XOR finds the lone number.

PROBLEM SOLVING

How to Solve Any Coding Problem + Java Traps



Say each step out loud in the interview.

1. **Read twice** : inputs, outputs, limits (n up to what?).
2. **Small examples** by hand, including empty and one element.
3. **Brute force** first, state its Big-O.
4. **Optimise** : which data structure removes repeated work?
5. **Code** with clear names. **Test** by tracing. **State** time and space.

Java traps interviewers love

```java
static void demo() {
    Integer a = 127, b = 127, c = 128, d = 128;
    System.out.println((a == b) + " " + (c == d) + " " + c.equals(d));   // Integer cache -128..127
    String s1 = new String("hi"), s2 = "hi";
    System.out.println((s1 == s2) + " " + s1.equals(s2));                // == compares references
    List<Integer> l = new ArrayList<>(List.of(1, 2, 3, 4));
    l.removeIf(v -> v % 2 == 0);
// safe removal while looping
    System.out.println(l);
}
```

```
output: true false true
false true
[1, 3]
```

Remember

Compare objects with `.equals()`. For `Integer` in collections never use `==`. Removing from a list inside a for-each loop throws **ConcurrentModificationException**.

PROBLEM SOLVING

Pattern Cheat Sheet: See X, Use Y

Most problems are a *known pattern* in disguise. Match the clue to the tool.

If you see...	Try this (Java)
sorted array, pair / target	two pointers, binary search
subarray / substring of size k, longest / shortest	sliding window
range sum queries	prefix sums
duplicates, counting, 'seen before?'	HashMap / HashSet
brackets, undo, next greater element	ArrayDeque as stack, monotonic stack
level by level, shortest steps	BFS with a queue
explore all, tree / graph paths	DFS / recursion
top K, K-th largest or smallest	PriorityQueue of size K
cycle in a list, middle node	slow & fast pointers
minimum value that works	binary search on the answer
all subsets, permutations, combinations	backtracking
count ways, min cost, overlapping subproblems	dynamic programming
intervals, earliest finish	sort + greedy
prerequisites / ordering	topological sort
groups, connected?	Union-Find
weighted shortest path	Dijkstra with PriorityQueue
appears twice except one	XOR

Tip

When two patterns fit, start with the simpler one. *Brute force first*, then the pattern that removes repeated work.

PROBLEM SOLVING

30-Day Plan & Interview Tips

Days	Topic	Practice
1–3	Java basics, Big-O, arrays	reverse, rotate, Kadane
4–6	Strings, hashing	anagram, two sum, group anagrams
7–9	Two pointers, sliding window	container water, longest substring
10–12	Recursion, sorting, binary search	merge sort, rotated array
13–15	Linked list, stack, queue	reverse list, valid brackets
16–19	Trees and BST	level order, LCA, validate BST
20–22	Heap, graphs	top K, islands, course schedule
23–25	Greedy, backtracking	intervals, subsets, N-Queens
26–28	Dynamic programming	stairs, coin change, LCS
29–30	Revision + mock interview	2 problems in 45 min, aloud

Interview tips

- ***Think aloud*** and ask clarifying questions first.
- Start with brute force, then improve step by step.
- ***Dry-run*** your code on one example before saying 'done'.
- Mention ***edge cases***: empty, one element, duplicates, negatives.
- Practise on paper: write Java by hand, no autocomplete!

Common mistakes

Mistake	Fix
off-by-one in loops	test first and last index
`int` overflow in sums	use `long`
`==` on Strings / Integers	use `.equals()`
forgetting empty input	check `a.length == 0` early

Secret

Keep a ***mistakes notebook***: re-solving a failed problem is worth three new ones.

PROBLEM SOLVING

References & Further Learning

These notes are written in my own words with *original chalk-style diagrams*. Every Java example was compiled and run.

Roadmaps

- **Scaler:** DSA in Java roadmap 2026: https://www.scaler.com/blog/?p=12466
- **NareshIT:** DSA with Java roadmap: https://nareshit.com/blogs/dsa-with-java-roadmap-what-to-learn-first-next-and-last

Learn and visualise

- **VisuAlgo:** https://visualgo.net/
- **Educative:** Data Structures for Coding Interviews in Java: https://www.educative.io/courses/data-structures-for-coding-interviews-in-java
- **Oracle Java docs:** java.util collections: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/package-summary.html
- **GeeksforGeeks:** https://www.geeksforgeeks.org/java/

Practise

- **LeetCode:** https://leetcode.com/ (start with Easy)
- **NeetCode:** https://neetcode.io/
- **HackerRank:** https://www.hackerrank.com/domains/java

More from us
Visit the **PrepPath E-books Store** website to explore more e-books for your learning journey. Shop now!